Case study — Securing a legal firm's web platform

> **Belona Cybersecurity** · 2026 Role: sole technical owner · Client: legal firm (Client A) Platform: Firebase / Google Cloud *Versión en español:* `espanol/caso-de-estudio-devsecops.md`
>
> **Read this first.** It's the executive summary of the whole engagement; the other documents develop each piece.

The problem

The firm had a website that **worked** — and that is exactly the most common and most deceptive situation: nothing was broken, so nothing looked urgent.

What it didn't have:

CONTROL	STATE BEFORE THE ENGAGEMENT
Admin panel authentication	username and password only
Access control	anyone with an account saw everything
Contact form	hosted by a third party
Backups	none
Monitoring	none
Data-protection compliance	no consent capture, no policy

And it held **client case files**: legal documents containing personal data.

None of this produces symptoms until it's too late. A site with no backups looks identical to one with backups — until the day you have to restore.

The objective

Reach a **defense-in-depth** posture — prevention, detection and recovery — covering the **AAA** model: authentication, authorization and accounting.

With two constraints from the business, which shaped every decision:

- **no added recurring cost**
- **no added third parties** inside the trust boundary

The second drove the work the most. Every external service that touches client data is one more system to watch, one more contract to review, and one more surface that can fail without warning.

What was implemented

Authentication — and a blocker that took work to understand

Two-step verification with a time-based code on the admin panel.

The real problem was a debugging one: enrolling the second factor failed every time. The cause was timing — enrollment was attempted while the session was not yet fully resolved, so the authenticated user did not yet exist as far as the system was concerned.

> Enrolling a second factor has to happen against a **fully resolved** session, not against one that is still finishing loading.

It's the kind of failure that appears in no guide: the configuration was correct, the order of execution was not.

Role-based authorization

Three roles with distinct permissions: **admin** (everything), **lawyer** (content, enquiries and case files) and **editor** (blog only).

Enforced in **two layers**:

1. **Server-side security rules** — the layer that actually decides
2. **The interface** — hides what the role cannot use

The second alone **is not security**: hiding a button does not prevent calling the function behind it. It exists so the experience makes sense, not to protect. Confusing the two is one of the most frequent mistakes in the field.

An in-house contact form

The external service was replaced by a server function that validates on the server, requires anti-abuse verification, stores the enquiry and sends the notification by mail.

The point isn't that it works the same: it's that the data of whoever writes no longer passes through a third party. At a law firm, someone's first message can contain sensitive information before any professional relationship exists.

Case files and private documents

Case-file management with document upload to **private storage**: no public tokenized links, authenticated download, audit fields, and size and type limits enforced by server-side rules.

A public tokenized link looks safe because it's long and unpredictable — but it **can be forwarded, it stays in browser history, and it never expires**. Whoever holds it is in.

Activity logging

A **write-only** audit collection: records can be created, nobody can edit or delete them, and only the admin can read. It records sensitive server-side actions, including the admin's own access.

A log the admin can delete is not a log. If the person with the most power can erase the evidence of what they did, the control does not exist.

Authorized penetration test, with remediation

A three-phase test: reconnaissance, authorization testing, and privilege-escalation attempts.

🔴 **Critical finding:** hosting was serving the project's version-control directory. Anyone could download the complete source code, its history, **and a repository access token** left behind in the configuration.

Cause: the configuration published the entire project folder, and the exclusion filter did not cover that directory. Nobody had put it there on purpose — it was there by default.

Fixed: those directories were excluded from hosting, security headers were added, and it was verified that they no longer respond.

✅ **What held:** role-based access control, the anti-abuse verification and the storage rules were tested live — as an anonymous visitor and as a user with a limited role — and **none gave way**. An authenticated editor could not grant themselves admin permissions or read data outside their role.

> A report that only lists what failed is incomplete. **Knowing what held is as important as knowing what gave way**, because it's what tells you where you don't need to spend.

Recovery

Four layers: point-in-time recovery (7 days), daily backups (14 days), document versioning (90 days), and a weekly off-site copy in a separate storage account.

See `disaster-recovery.md` — the detail is there, including a later finding: one of those layers was documented as implemented and had never been enabled.

Detection

- **File integrity monitoring:** a cryptographic fingerprint of the production site's files, compared automatically every six hours, alerting if anything changes without a deployment.
- **Log-based alerts** on system errors.
- **Supply chain:** automatic dependency review on every change. Went from **20 vulnerabilities (1 critical, 5 high) to 8 moderate**.

Integrity monitoring answers one concrete question: *did anyone modify the site without me deploying it?* Without it, an attacker who injects code into a page can sit there for months unnoticed.

Secrets management

External service keys were moved out of a file in the project and into a **managed encrypted store**.

A configuration file holding credentials **is still a file on disk**: a backup copies it, a careless `git add` uploads it, anyone with access to the machine reads it. The encrypted store removes the file from the equation.

Regulatory compliance

Consent capture with stored proof, an international-transfer clause in the privacy policy, and a documented roadmap of what remains at the organizational level.

What's pending was written down, not omitted. A compliance report that lists only what was done is advertising.

Documentation produced

- **PASTA threat model** (stages 3–7): decomposition, threat analysis, mapping to known vulnerability classes, attack trees with the validated controls, and a residual-risk matrix.
- **NIST-based incident response playbooks.**
- **A staff training program**, aligned to CIS Control 14.

Results

OUTCOME	WHAT IT MEANS
Full AAA	authentication, authorization and accounting
Defense in depth	prevention, detection and recovery
One critical finding	source code and credential exposure, found and fixed **before** being exploited
Supply chain	from 20 vulnerabilities to 8 moderate
No third parties added	and no new recurring cost

What this case teaches

1. Working does not mean secure. The platform was operating normally while serving its own source code and a credential to anyone who knew to ask.

2. Controls are verified, not declared. The critical finding appeared on testing, not on reviewing the configuration. And months later, another control documented as active turned out not to be.

3. Knowing what held is worth as much as knowing what gave way. The privilege-escalation tests found nothing — and that negative result, obtained by genuinely trying, is useful information.

From the technical portfolio of Belona Cybersecurity. Real work on a production platform, performed under authorization from the owner.